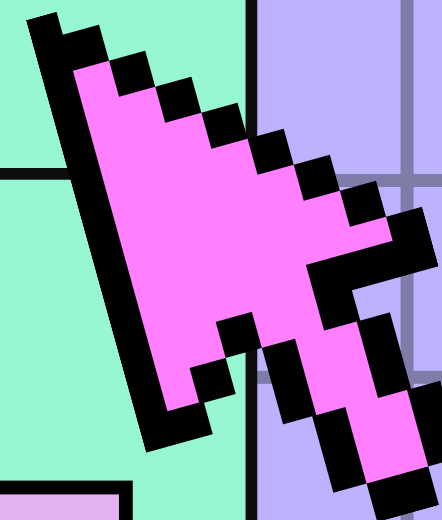


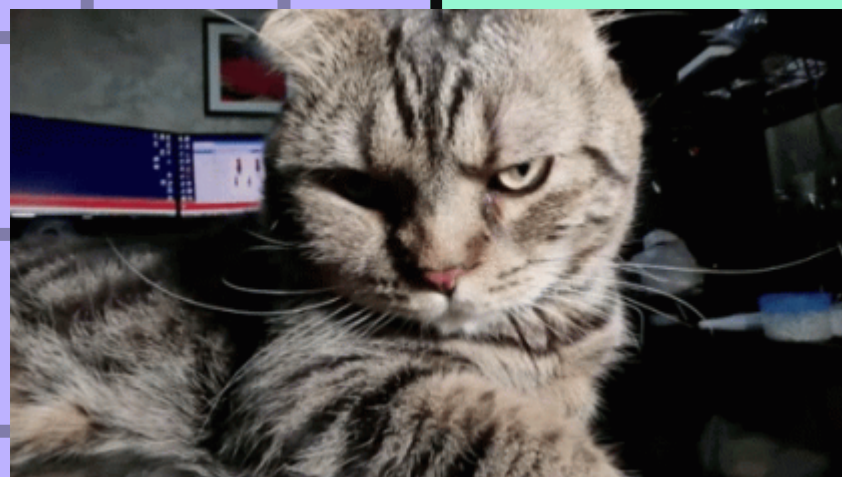


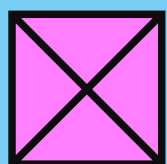
CC3501



AUXILIAR E

PROYECCIONES + TEXTURAS





AGENDA

VOLUMEN DE
VISTA

TIPOS DE
PROYECCIONES

FRAGMENT
SHADER

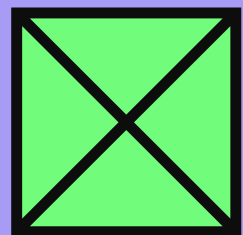


PROYECCIONES
EN EL PIPELINE

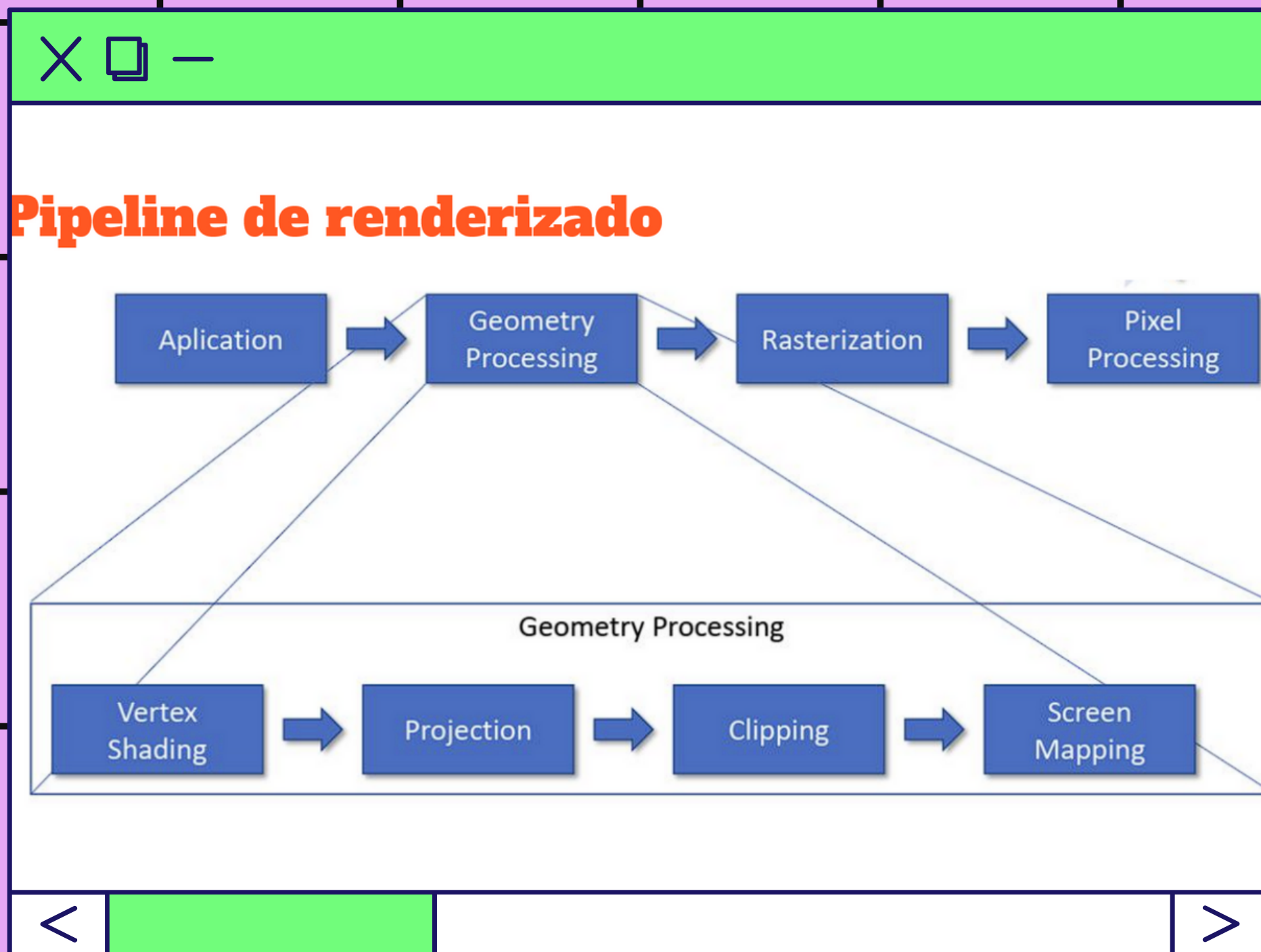
PROYECCIONES

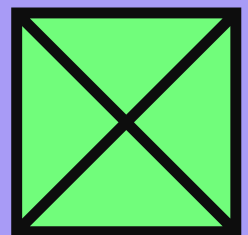
TEXTURAS EN
EL PIPELINE

COORDENADAS
DE TEXTURA

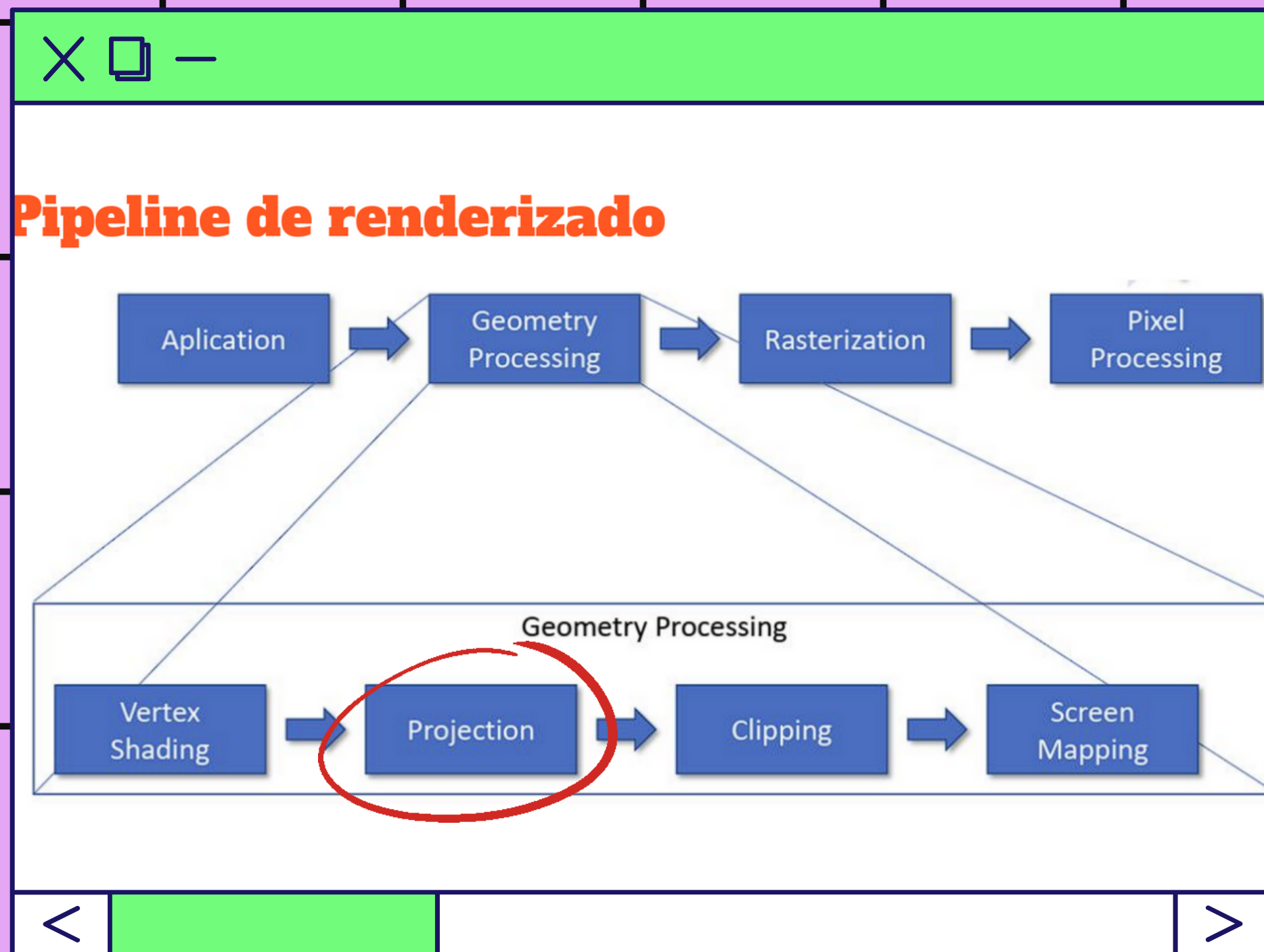


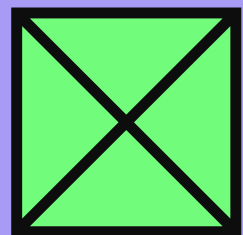
HAGAMOS MEMORIA...





HAGAMOS MEMORIA...

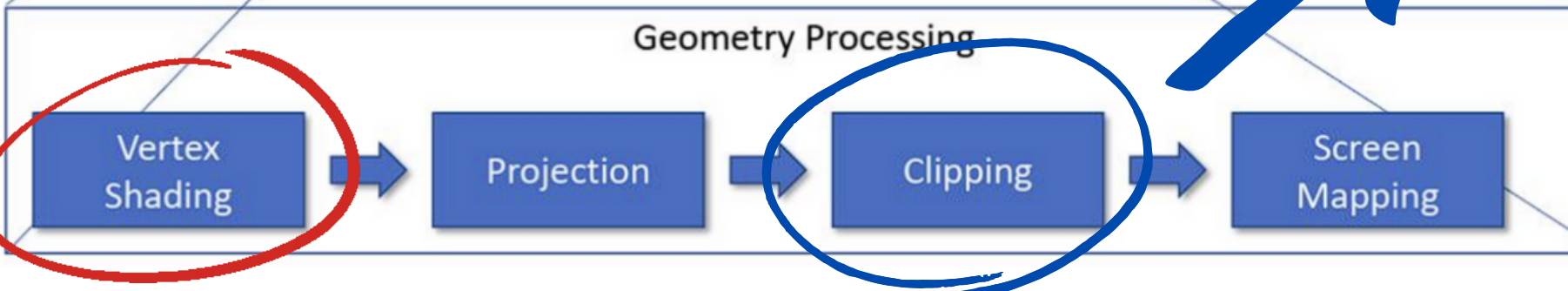




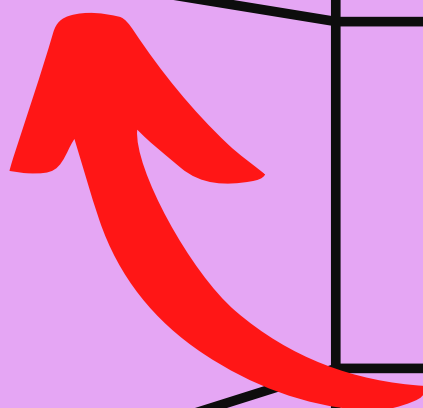
HAGAMOS MEMORIA...



pipeline de renderizado

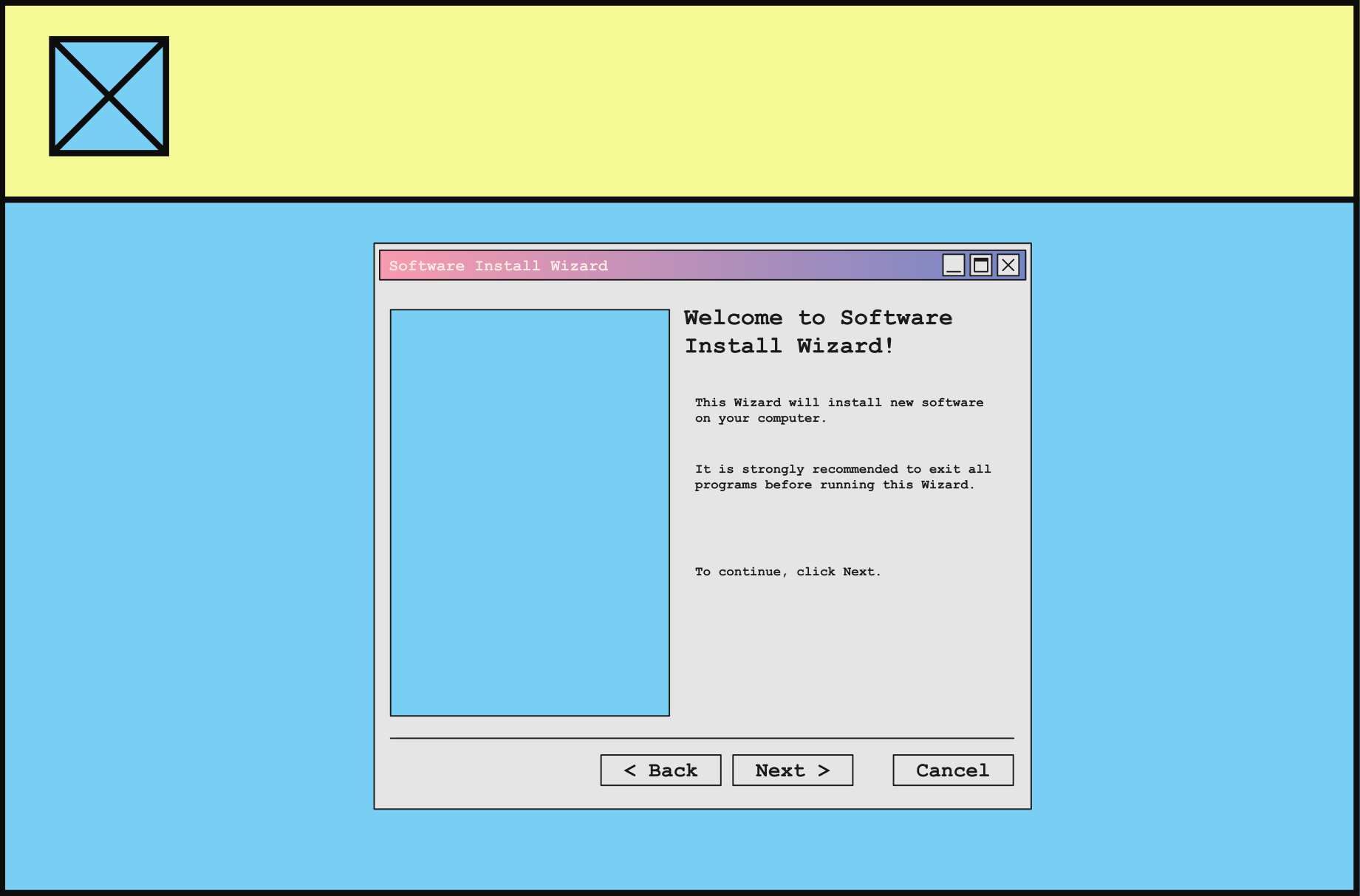


donde se aplican las transformaciones



donde se recorta la geometría que no se ve



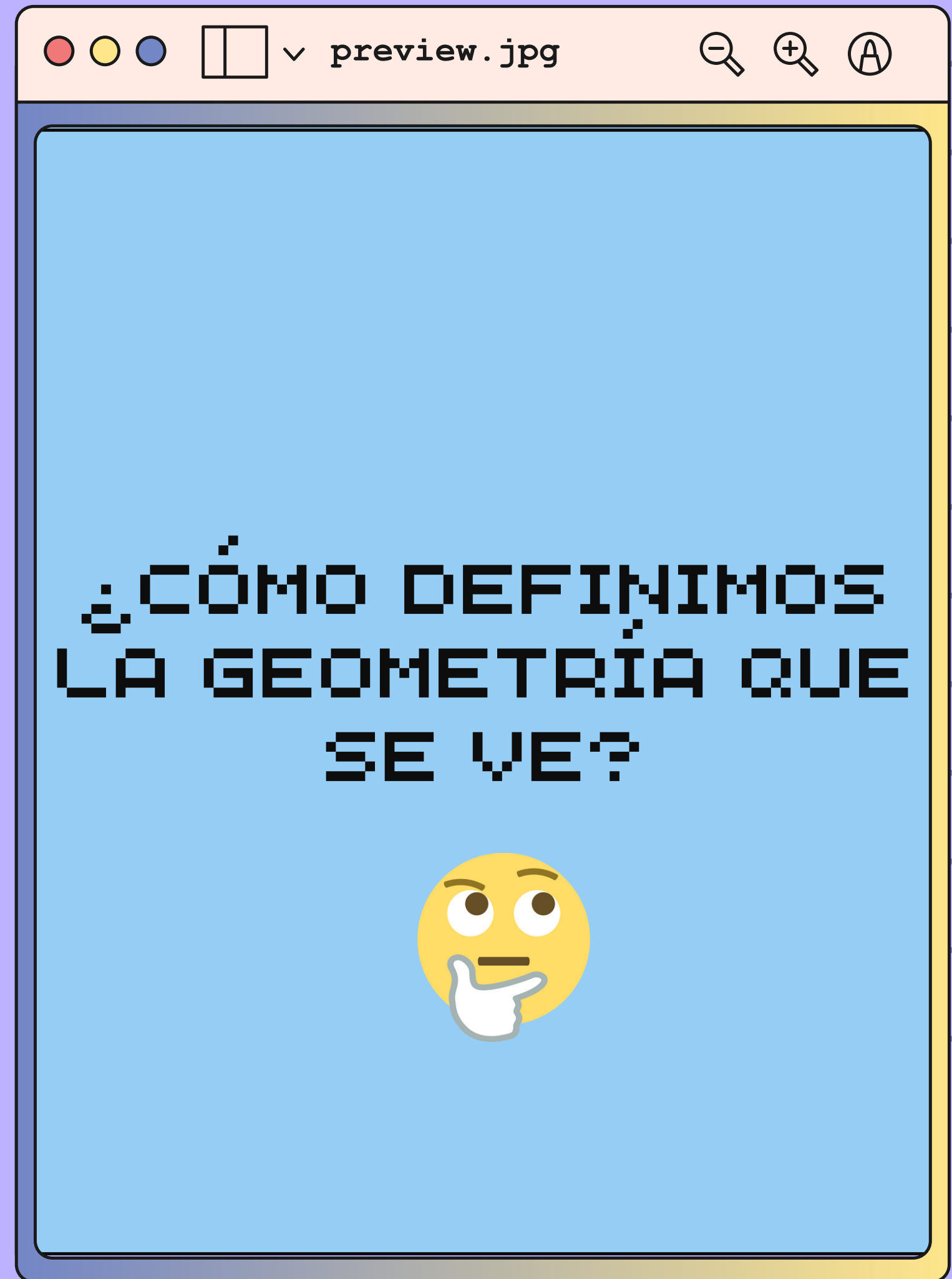
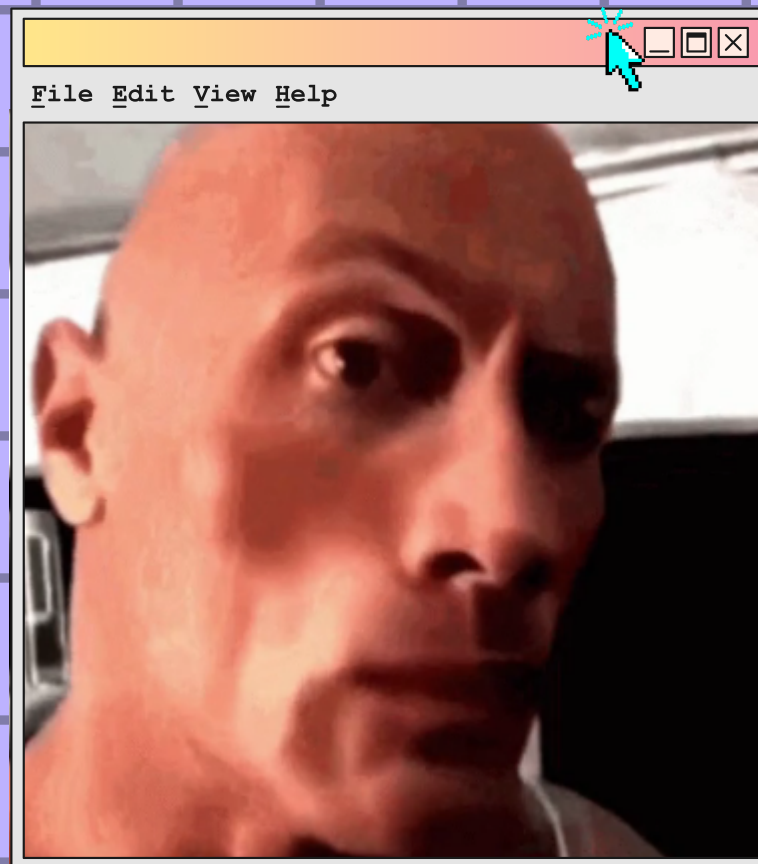




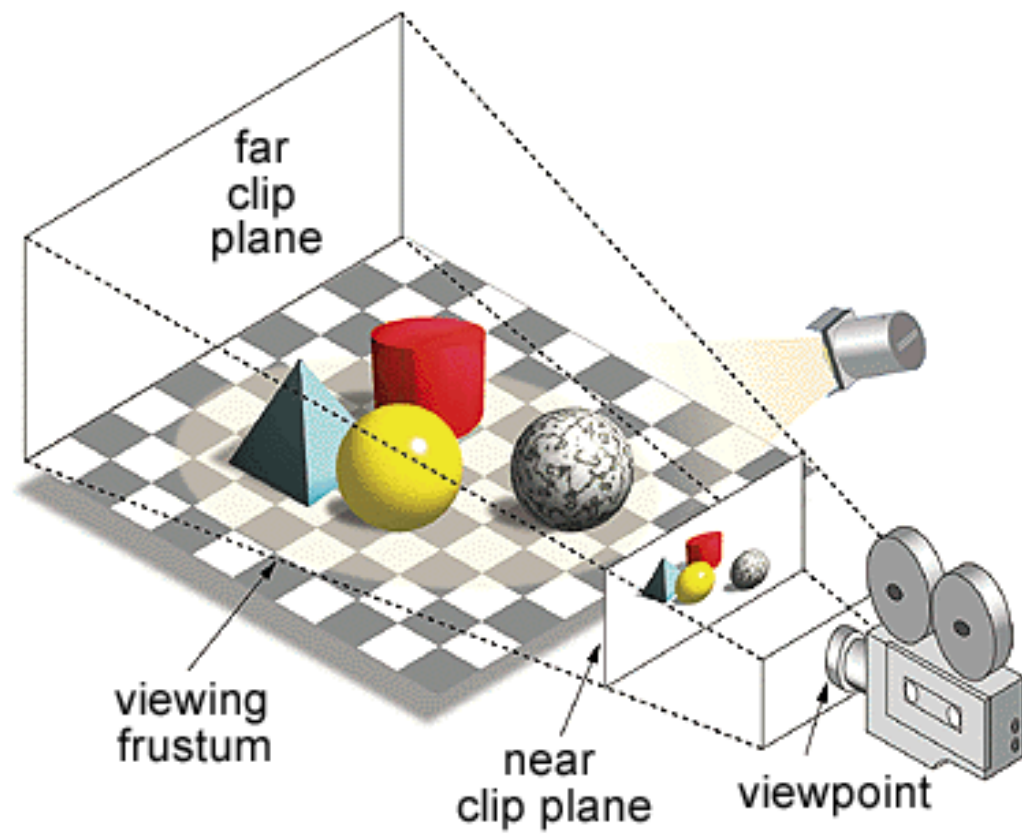
¿QUÉ
FALTA
PARA
PASAR AL
CLIPPING?



¡DEFINIR LA
GEOMETRÍA QUE SE
VE!



¡CON EL VOLUMEN DE
VISTA!



preview.jpg



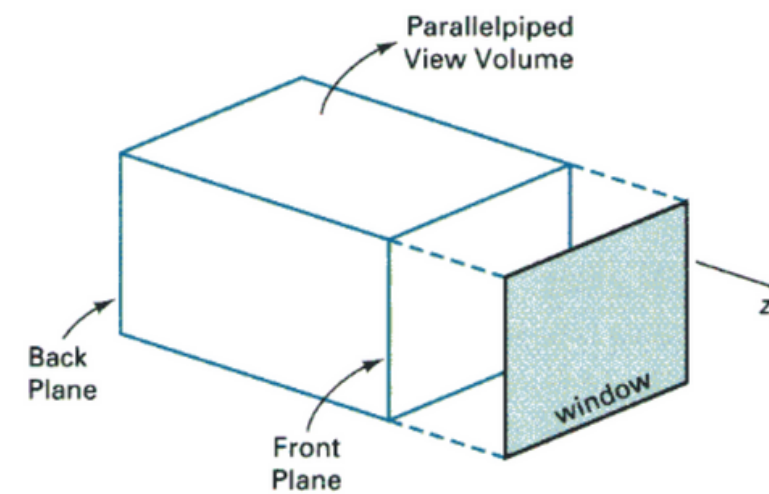
¿CÓMO DEFINIMOS
LA GEOMETRÍA QUE
SE VE?



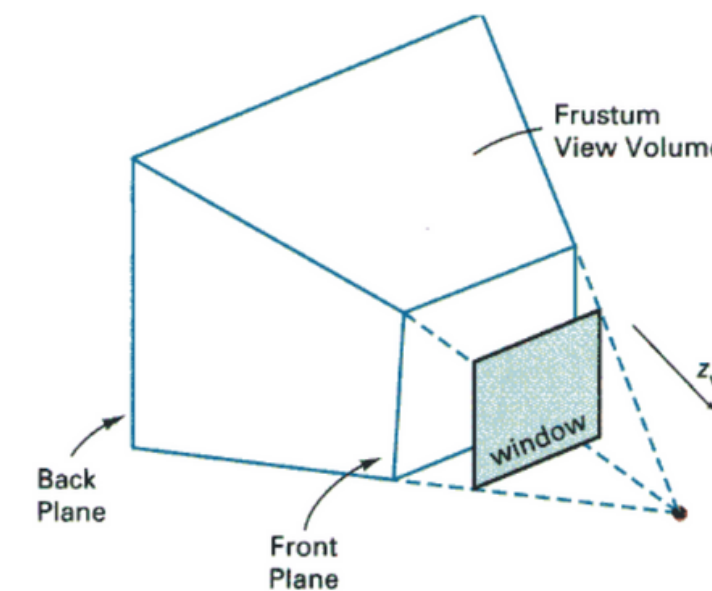
VOLUMEN DE VISTA



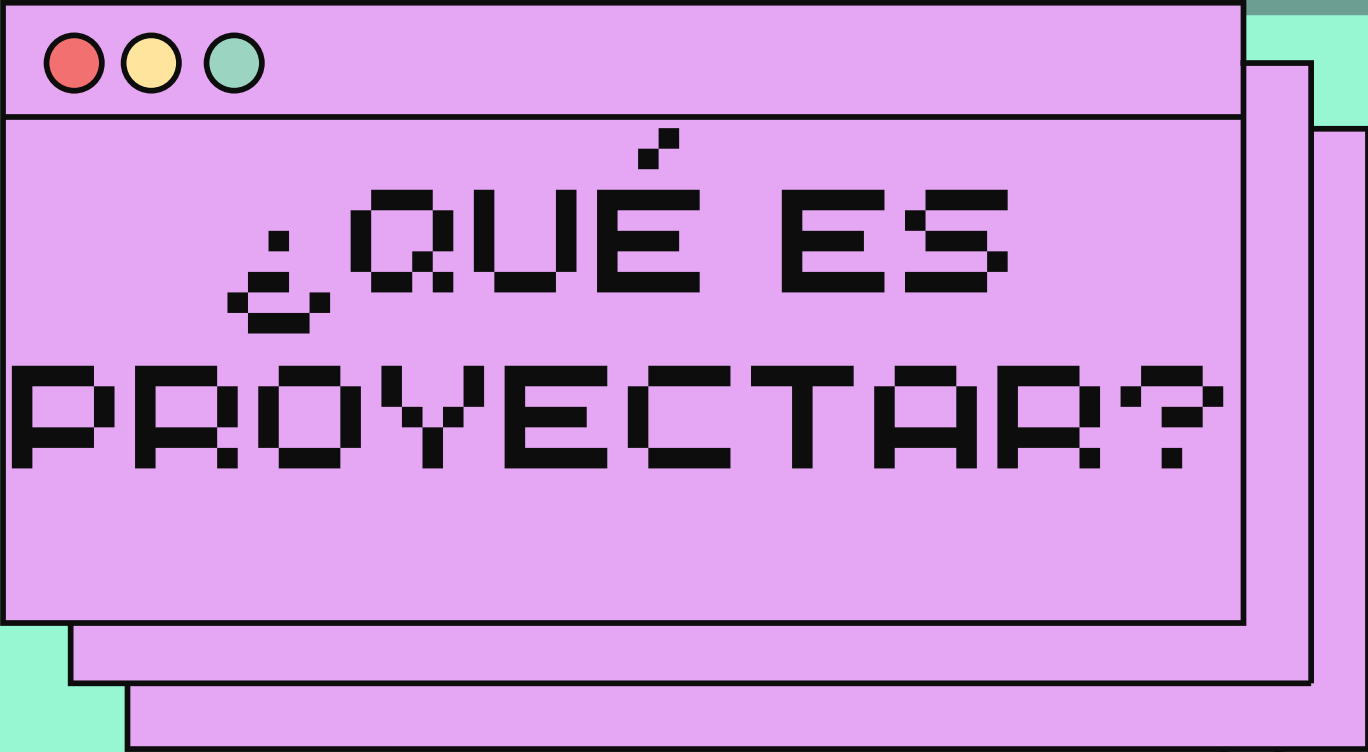
El volumen de vista encierra toda la geometría visible en un paralelepípedo o frustum.



Parallel Projection



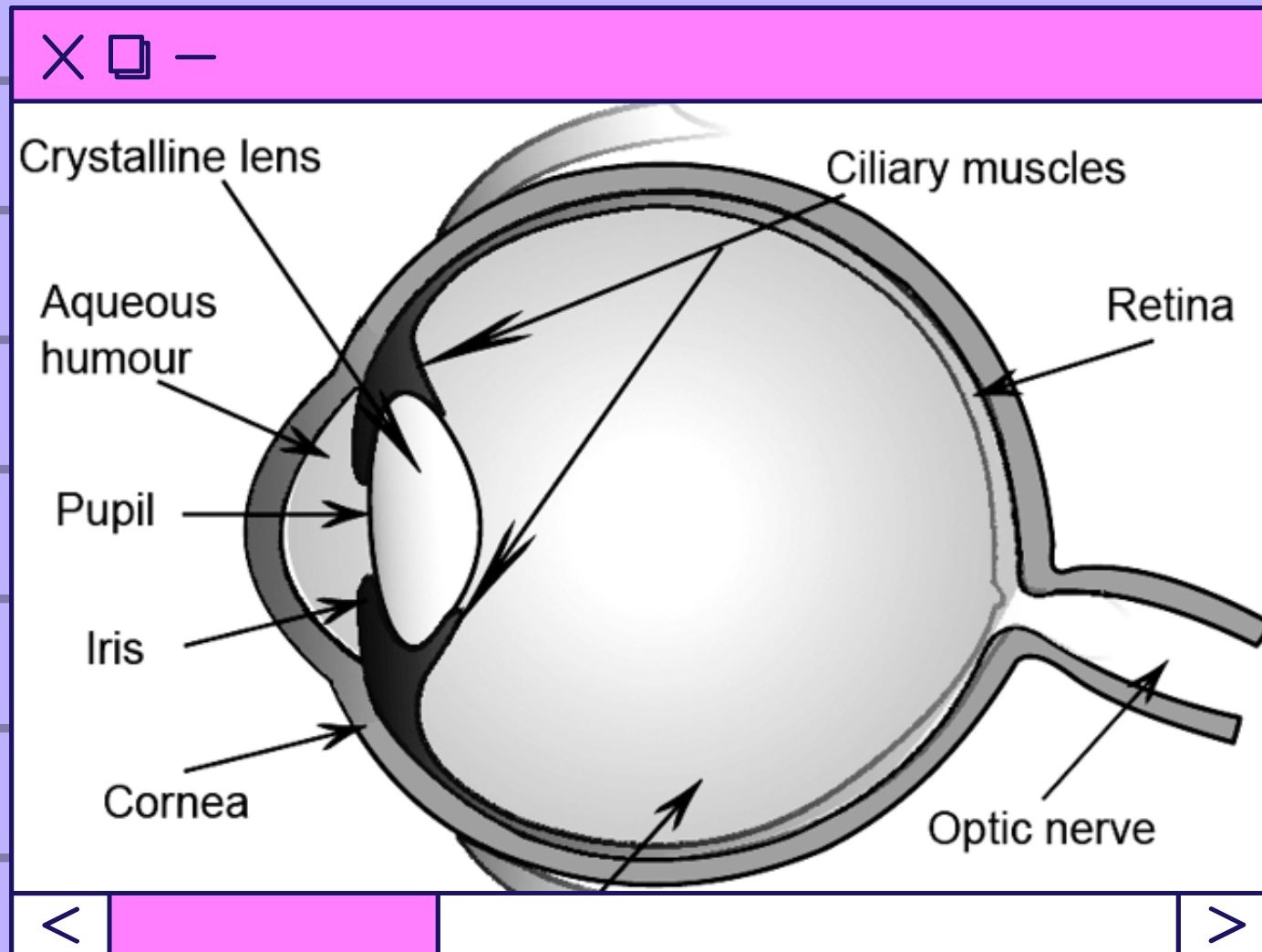
Perspective Projection



¿QUÉ ES PROYECTAR?

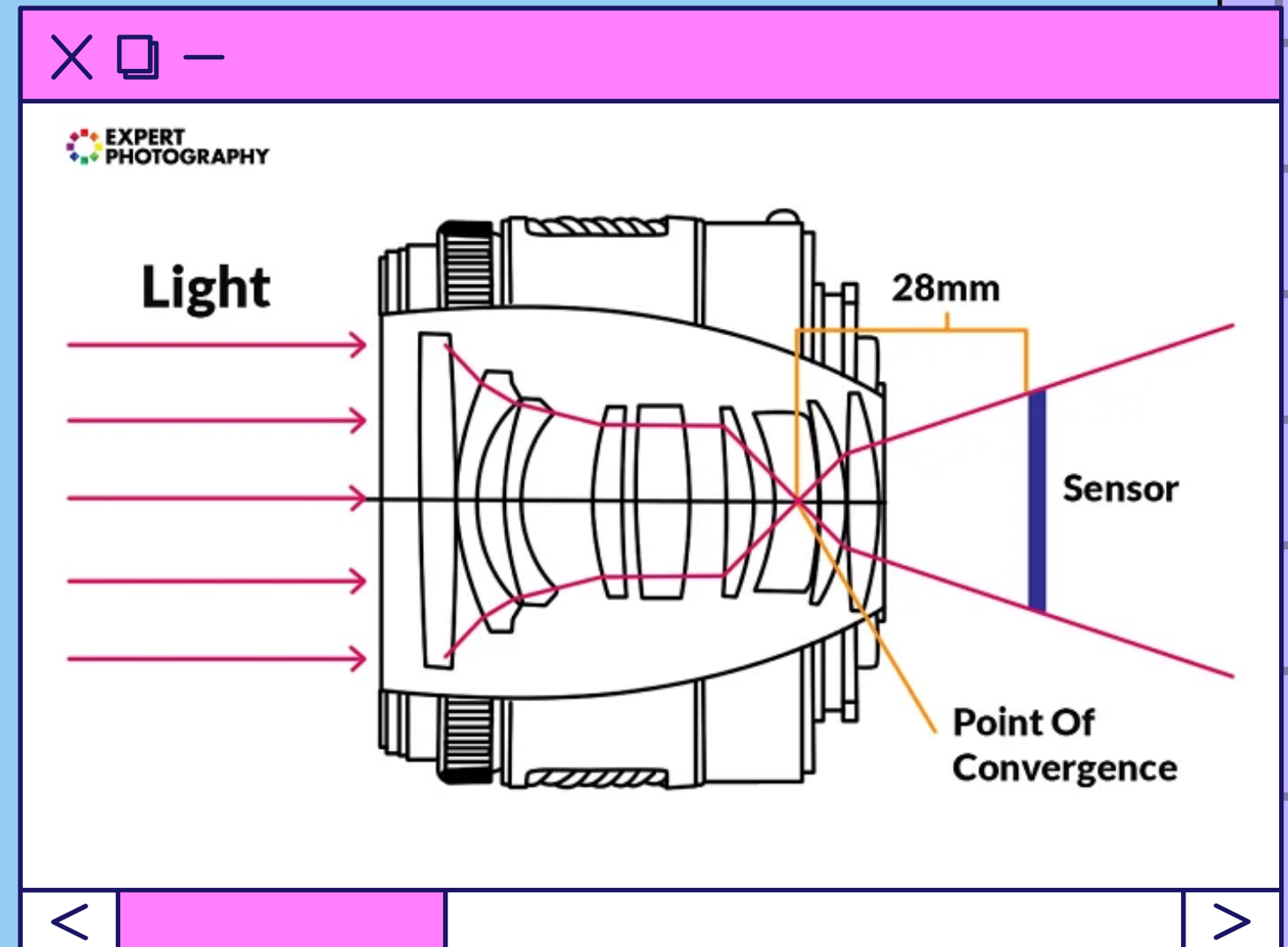
Proyectar:

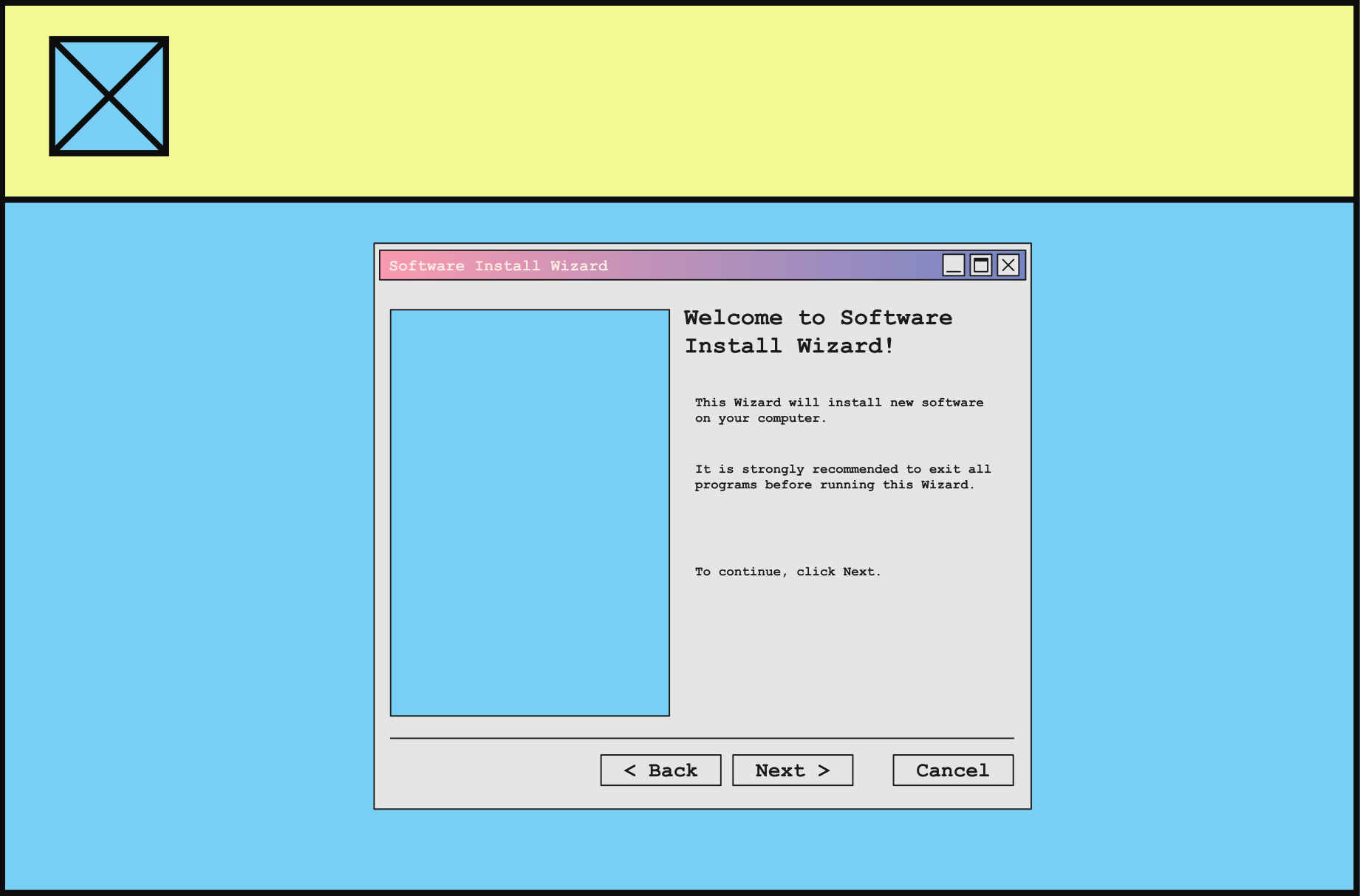
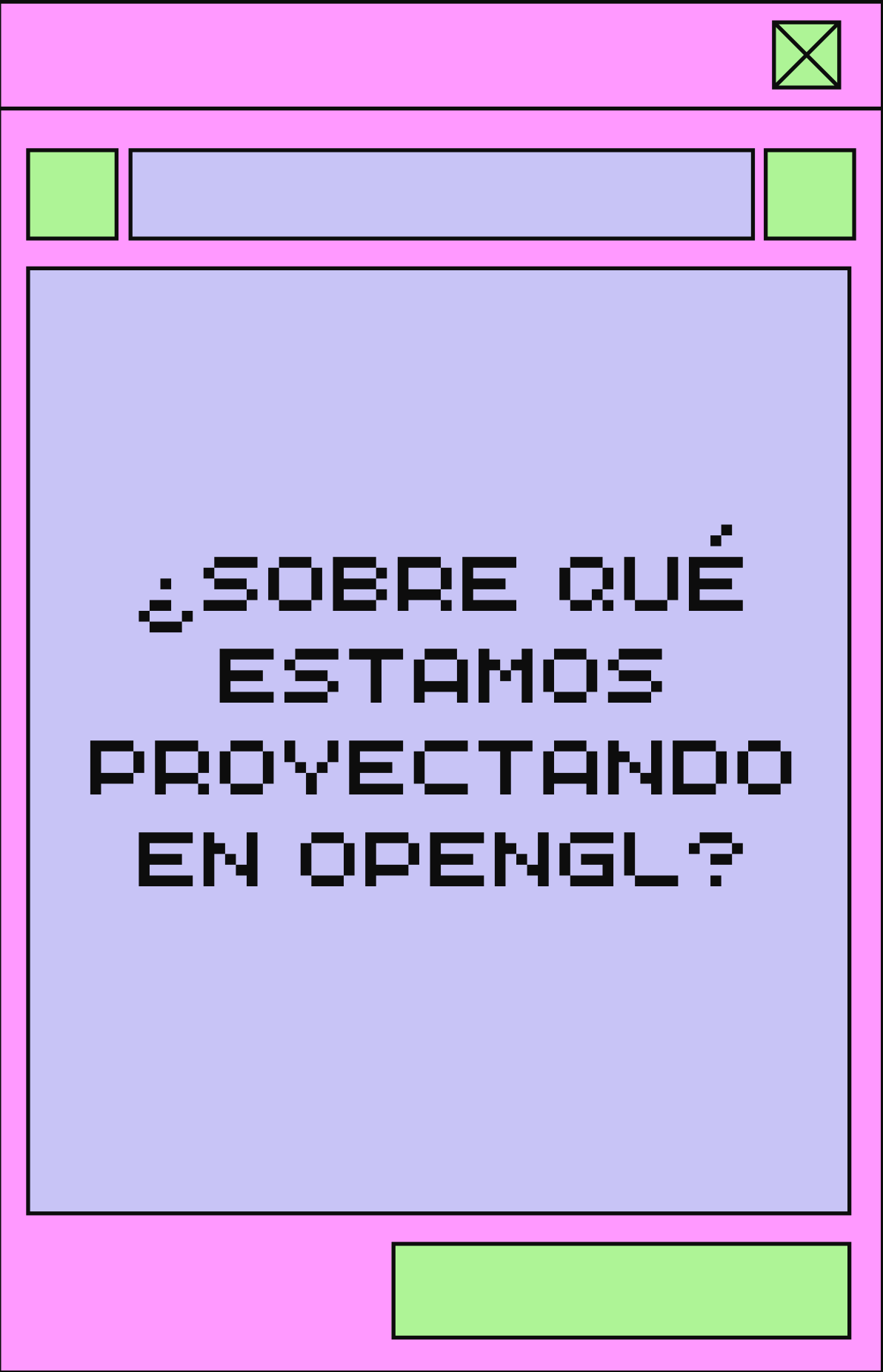
- Hacer visible sobre un cuerpo o una superficie la figura o la sombra de otro.
- Geom. Trazar líneas rectas desde todos los puntos de un sólido u otra figura, según determinadas reglas, para obtener su representación en una superficie.

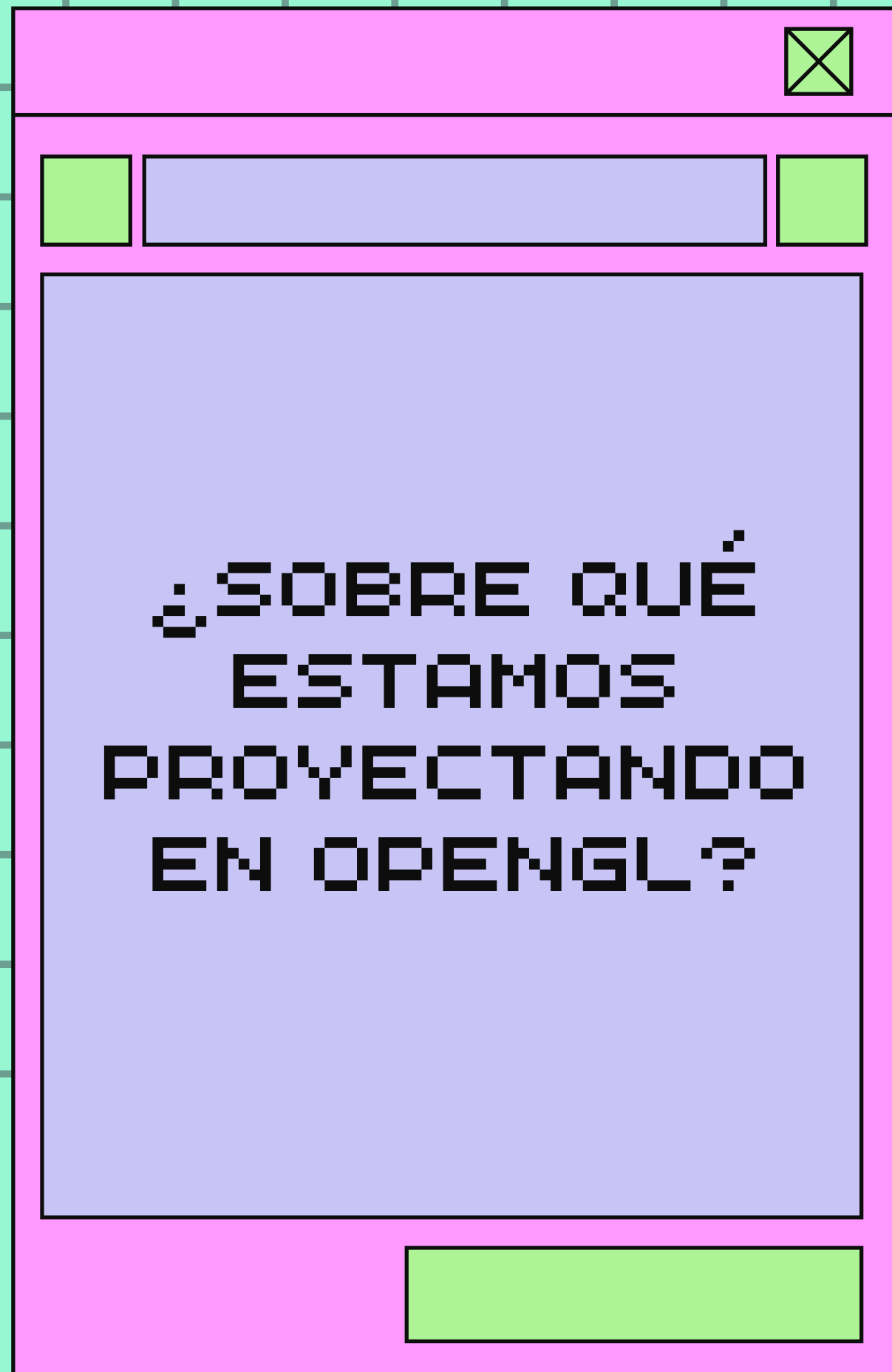


Las proyecciones están en muchos lugares de nuestro mundo. El ojo proyecta el mundo directamente la retina. Las cámaras lo proyectan al sensor.

¿SOBRE UNA SUPERFICIE?







¿SOBRE QUÉ
ESTAMOS
PROYECTANDO
EN OPENGGL?

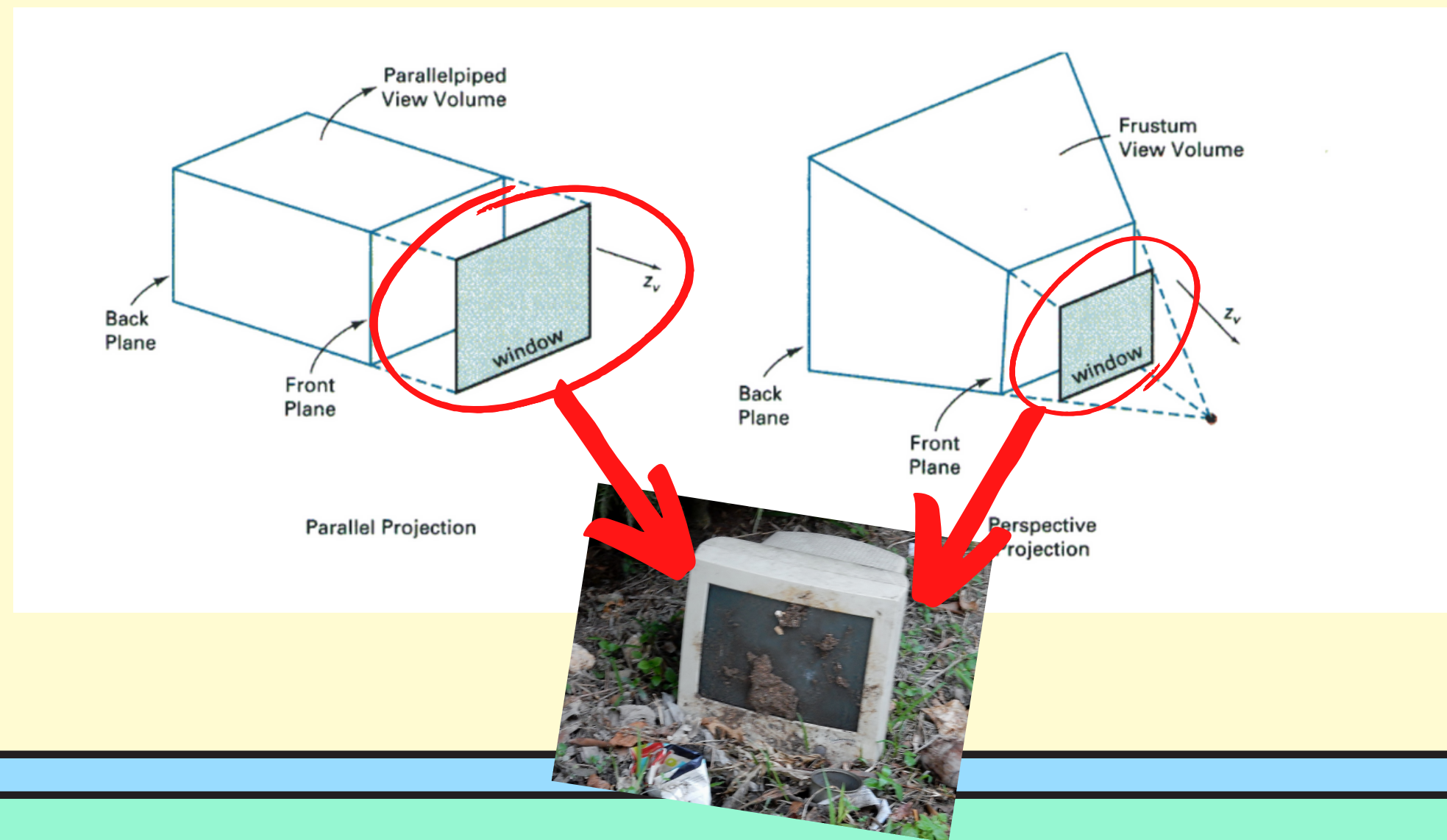


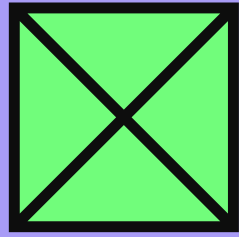
¿SOBRE LA
VENTANA!



VOLUMEN DE VISTA

El volumen de vista encierra toda la geometría visible en un paralelepípedo o frustum.

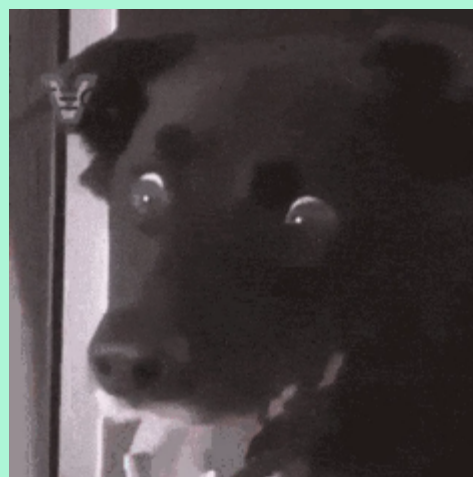




¡PLOT TWIST!



Las proyecciones son las últimas transformaciones que se aplican en el **vertex shader**... ¡Estuvieron ahí todo este tiempo!

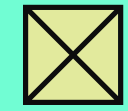


```
# Using the same view and projection matrices in the whole application
projection = tr.perspective(45, float(width)/float(height), -0.1, 100)
glUniformMatrix4fv(glGetUniformLocation(mvpPipeline.shaderProgram, "proj

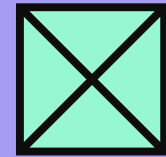
view = tr.lookAt(
    np.array([5,5,7]),
    np.array([0,0,0]),
    np.array([0,0,1])
)
glUniformMatrix4fv(glGetUniformLocation(mvpPipeline.shaderProgram, "view"
```

```
vertex_shader = """
#version 330

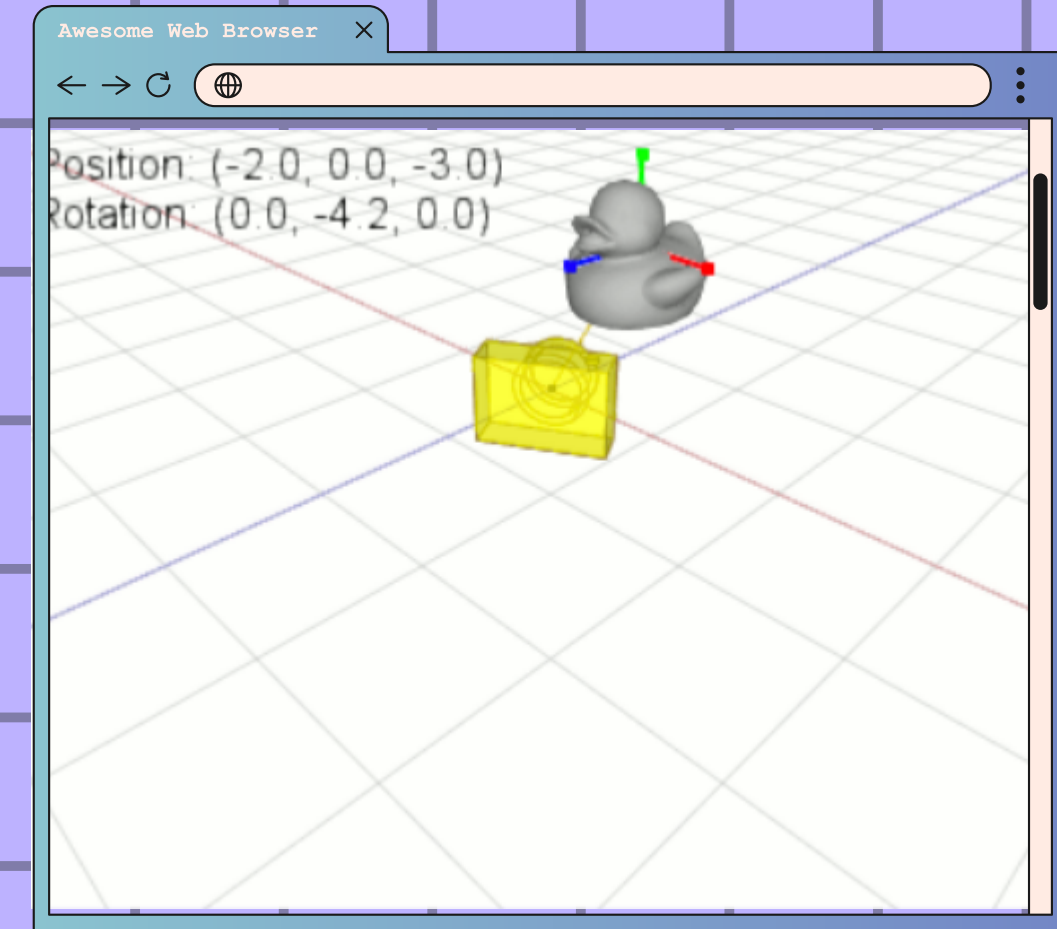
uniform mat4 projection;
uniform mat4 view;
uniform mat4 transform;
in vec3 position;
in vec3 color;
out vec3 newColor;
void main()
{
    gl_Position = projection * view * transform * vec4(position, 1.0f);
    newColor = color;
}
"""
```

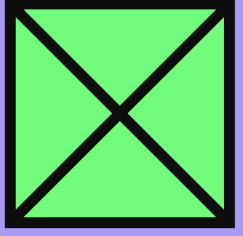



VARIABLES IMPORTANTES

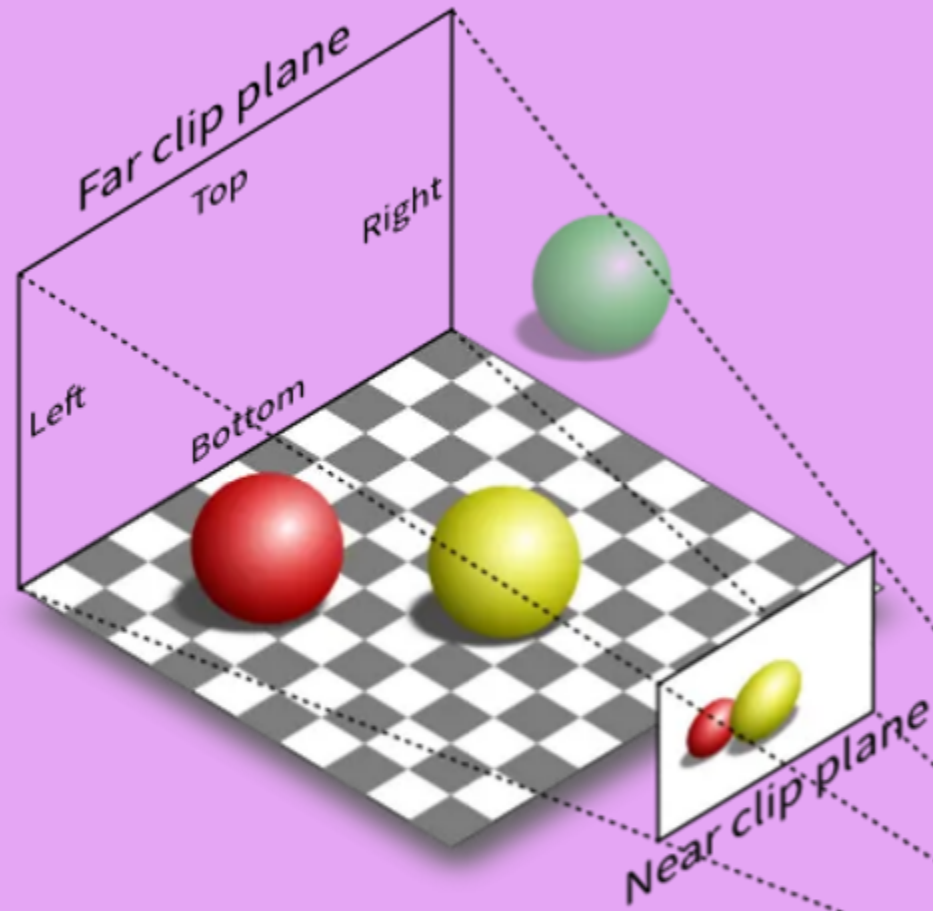


- view: transformación que sitúa la cámara en su lugar (rotación y traslación).
- projection: transformación que da la forma al volumen de vista.



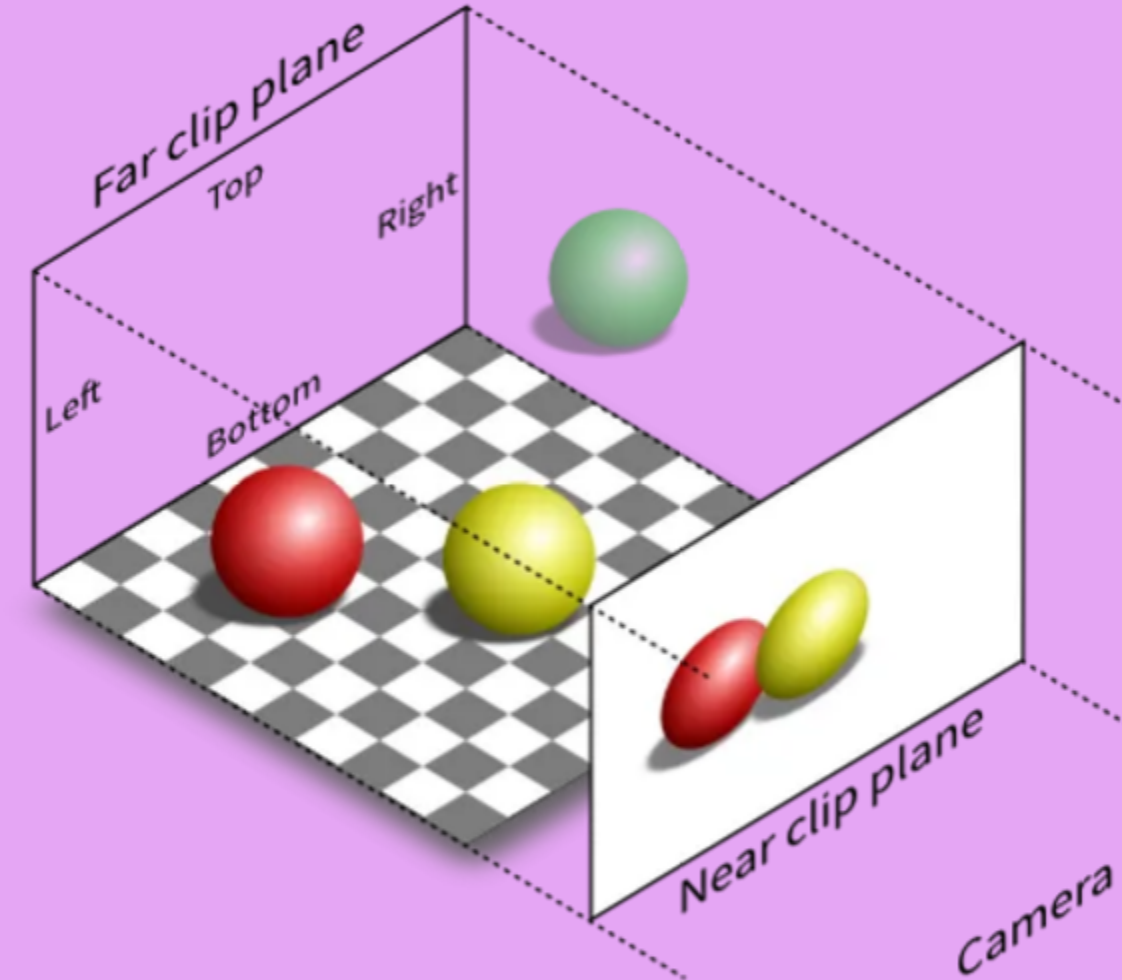


TIPOS DE PROYECCIONES



Perspective projection (P)

```
def perspective(fovy, aspect, near, far):  
    halfHeight = np.tan(np.pi * fovy / 360) * near  
    halfWidth = halfHeight * aspect  
    return frustum(-halfWidth, halfWidth, -halfHeight, halfHeight, near, far)
```

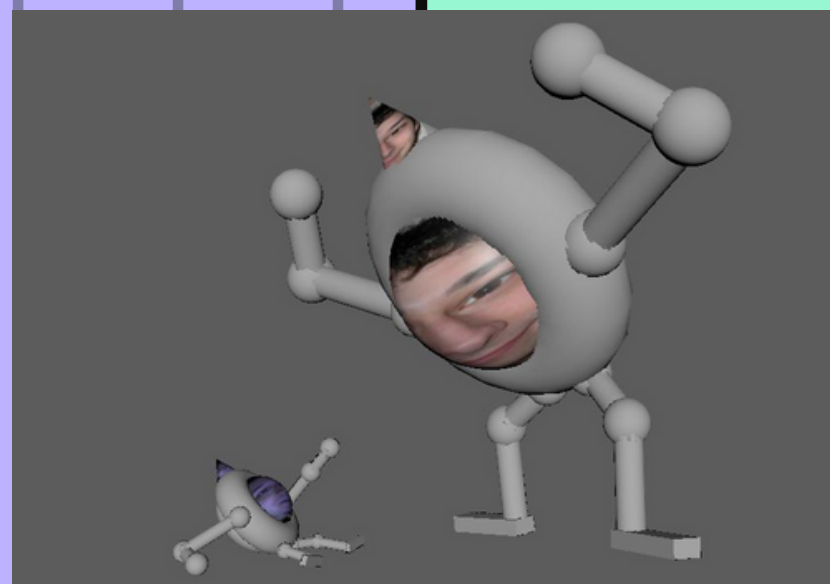
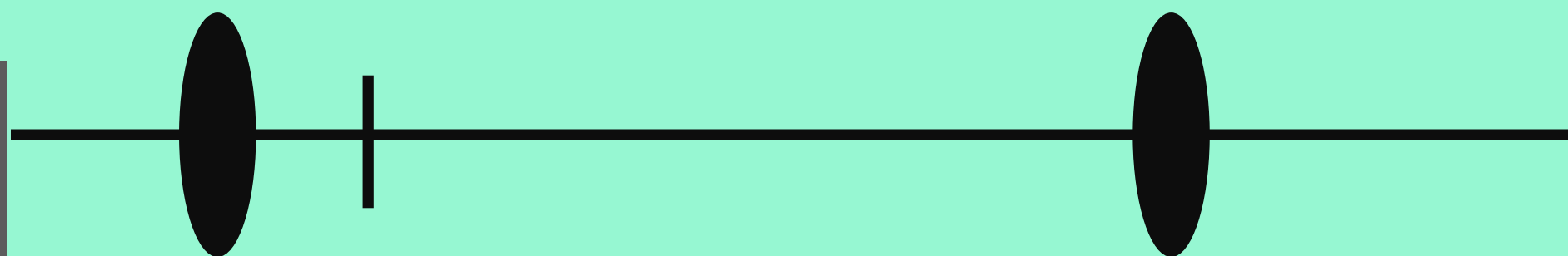


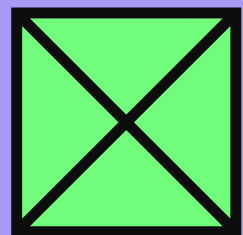
Orthographic projection (O)

```
def ortho(left, right, bottom, top, near, far):  
    r_l = right - left  
    t_b = top - bottom  
    f_n = far - near  
    return np.array([
```

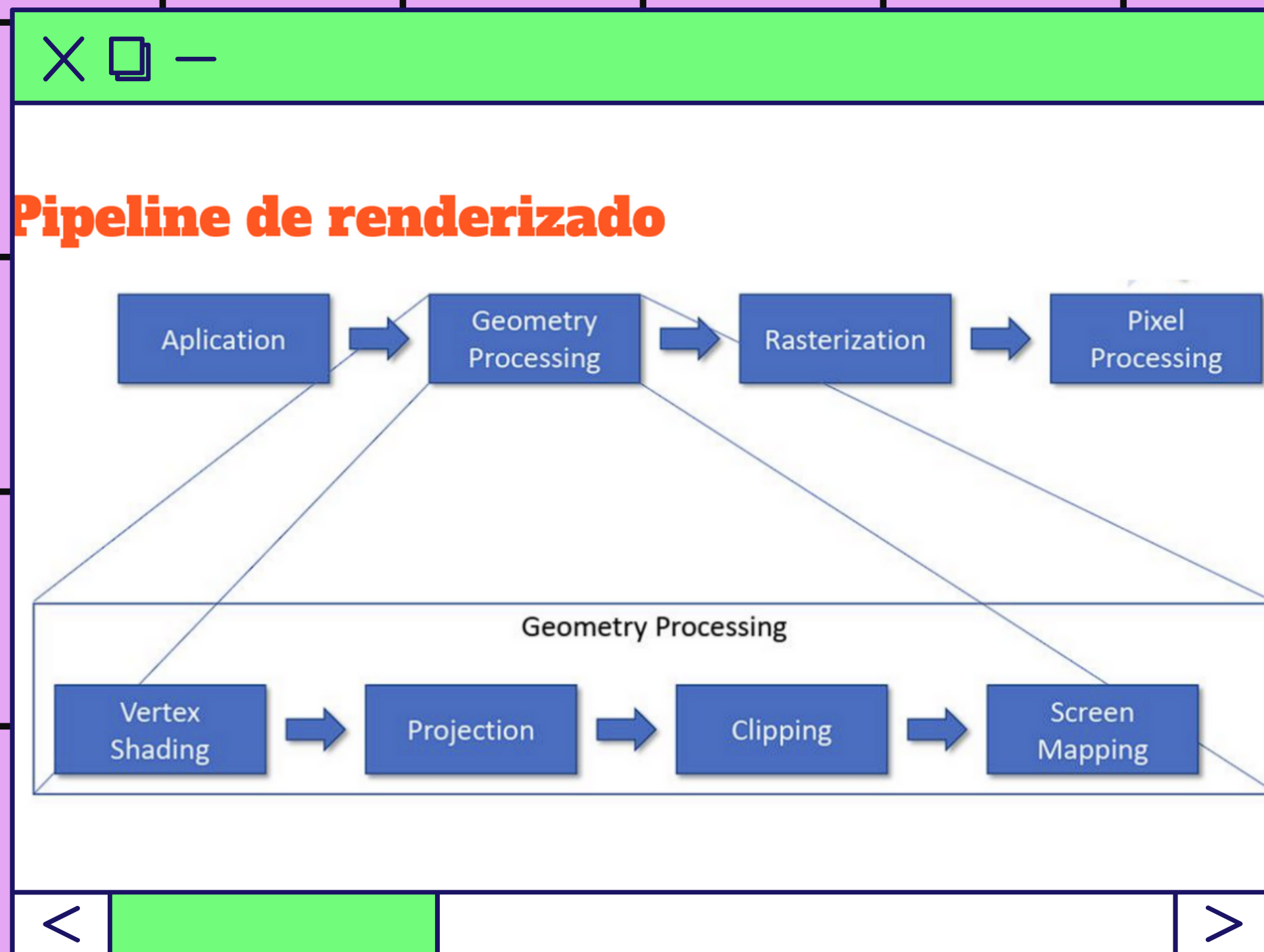


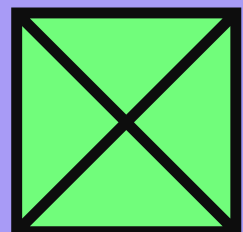
TEXTURE



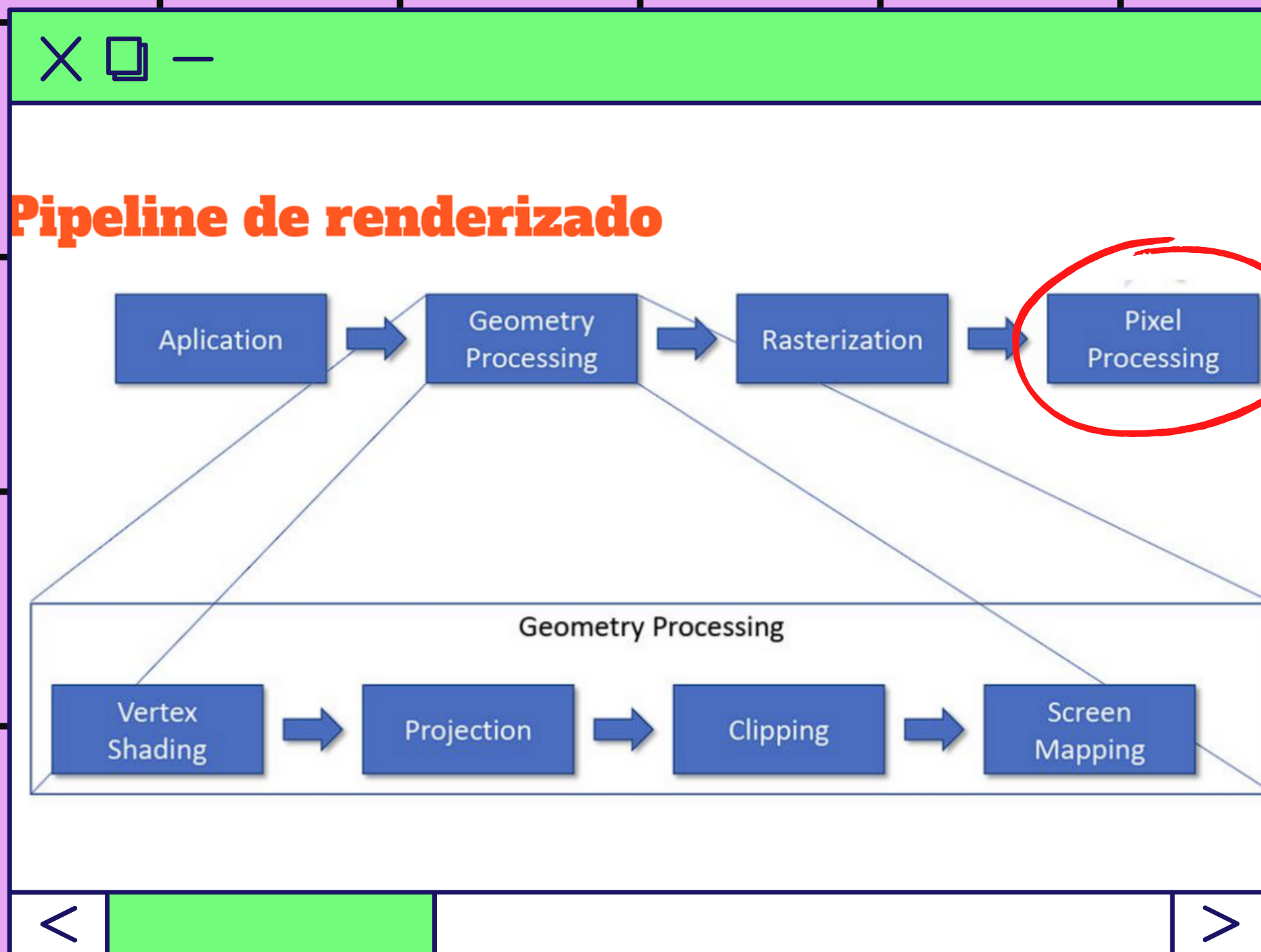


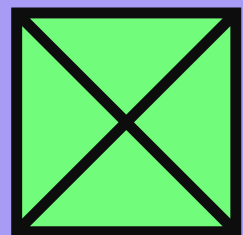
HAGAMOS MEMORIA... (de nuevo)



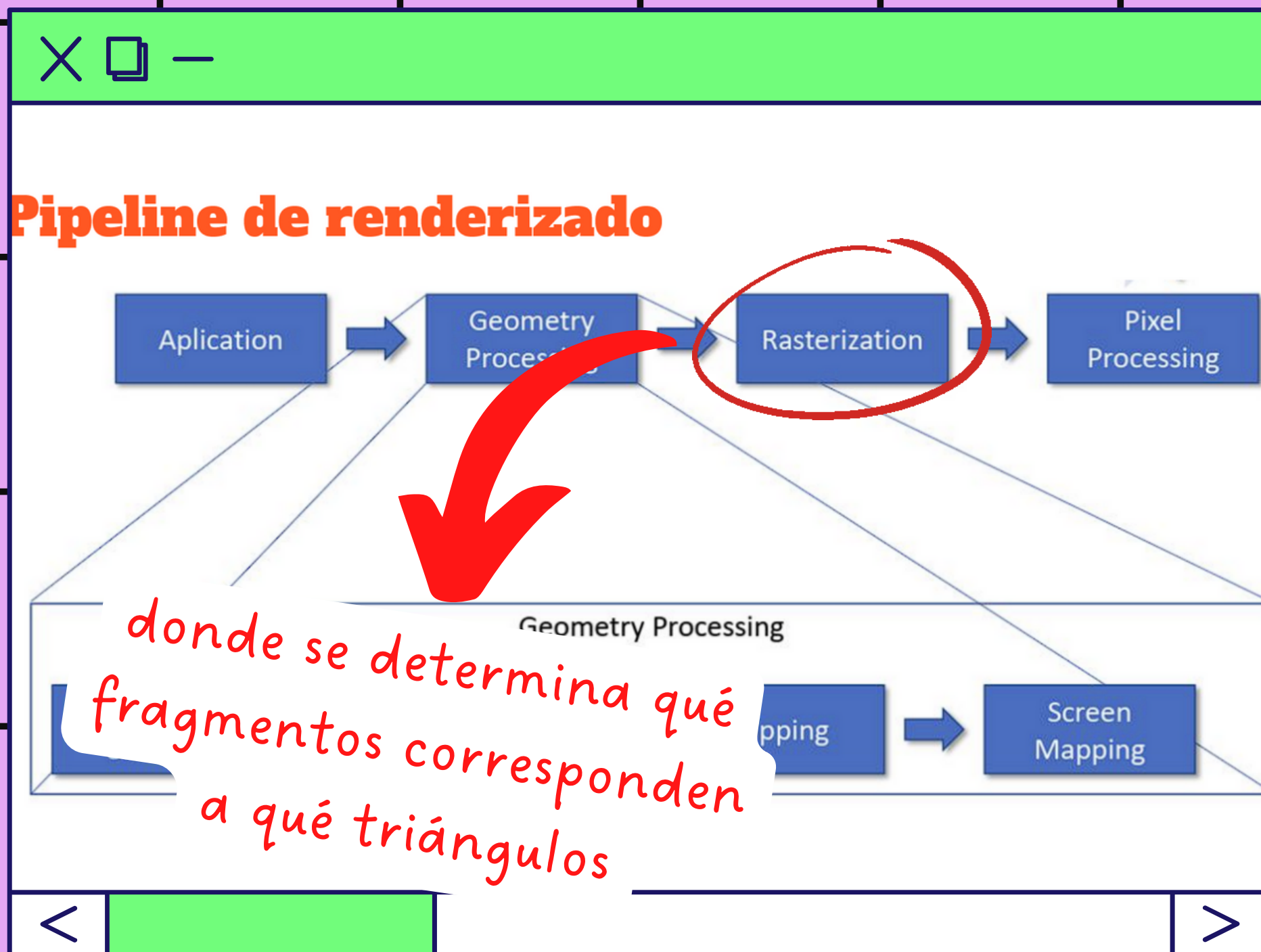


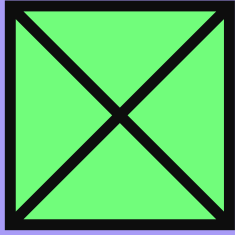
HAGAMOS MEMORIA... (de nuevo)





HAGAMOS MEMORIA... (de nuevo)





FRAGMENT SHADER

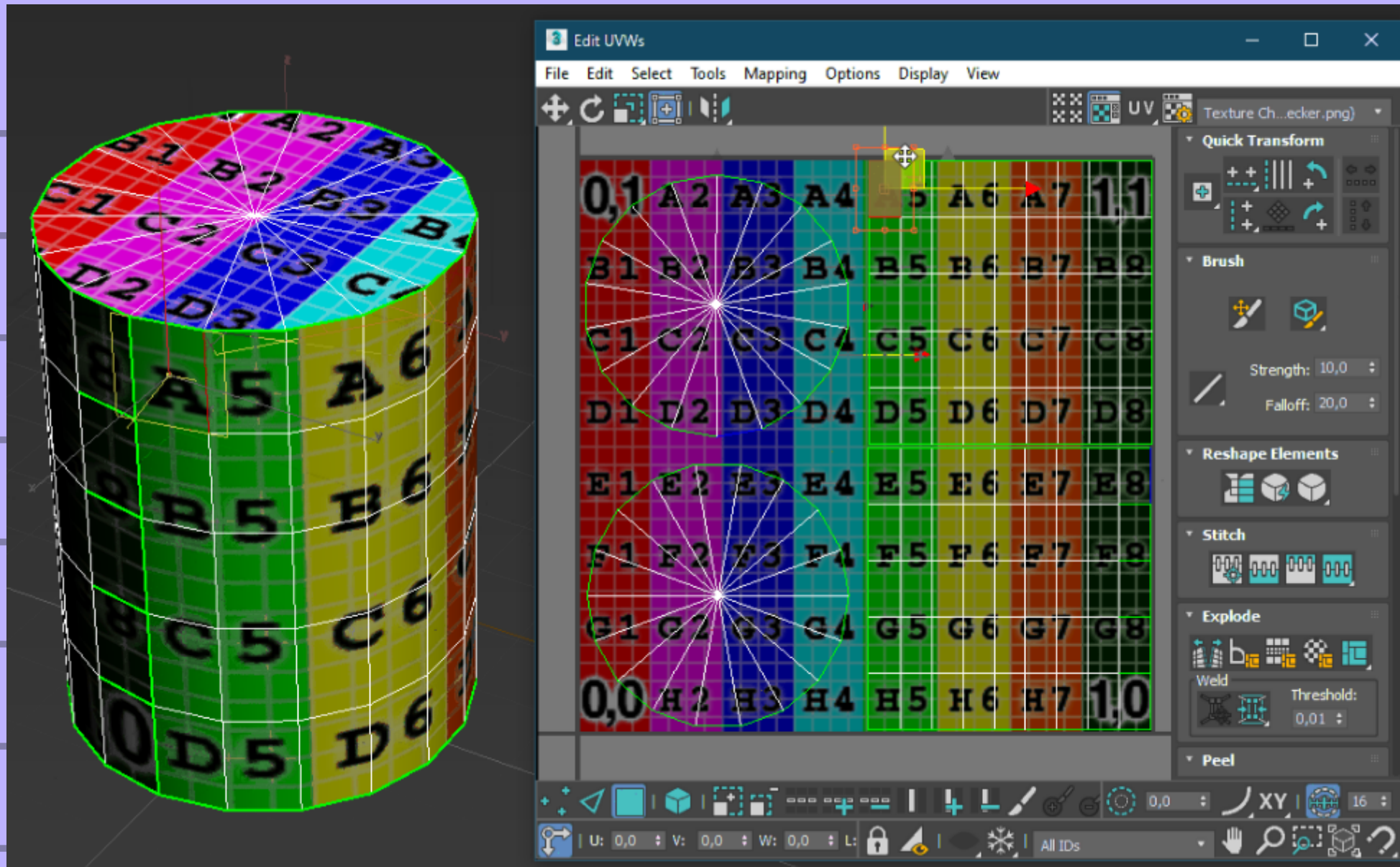


```
fragment_shader = ""  
#version 330  
  
uniform sampler2D samplerTex;  
  
in vec2 outTexCoords;  
  
out vec4 outColor;  
  
void main()  
{  
    outColor = texture(samplerTex, outTexCoords);  
}  
""
```



También conocido como Pixel Shader, opera sobre cada candidato a píxel.

Para el Texture Mapping, cada fragmento busca su color en una imagen, por medio de un mapa de coordenadas.



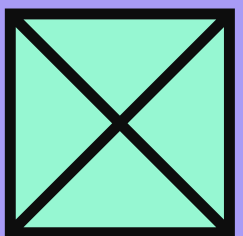
El mapa UV asocia coordenadas a cada triángulo en el espacio de la textura.

Ahora, además de definir la posición de cada vértice, es necesario definir las coordenadas 2D en las que se define su color.

Además, el VAO debe estar enterado de dónde encontrar estas coordenadas.

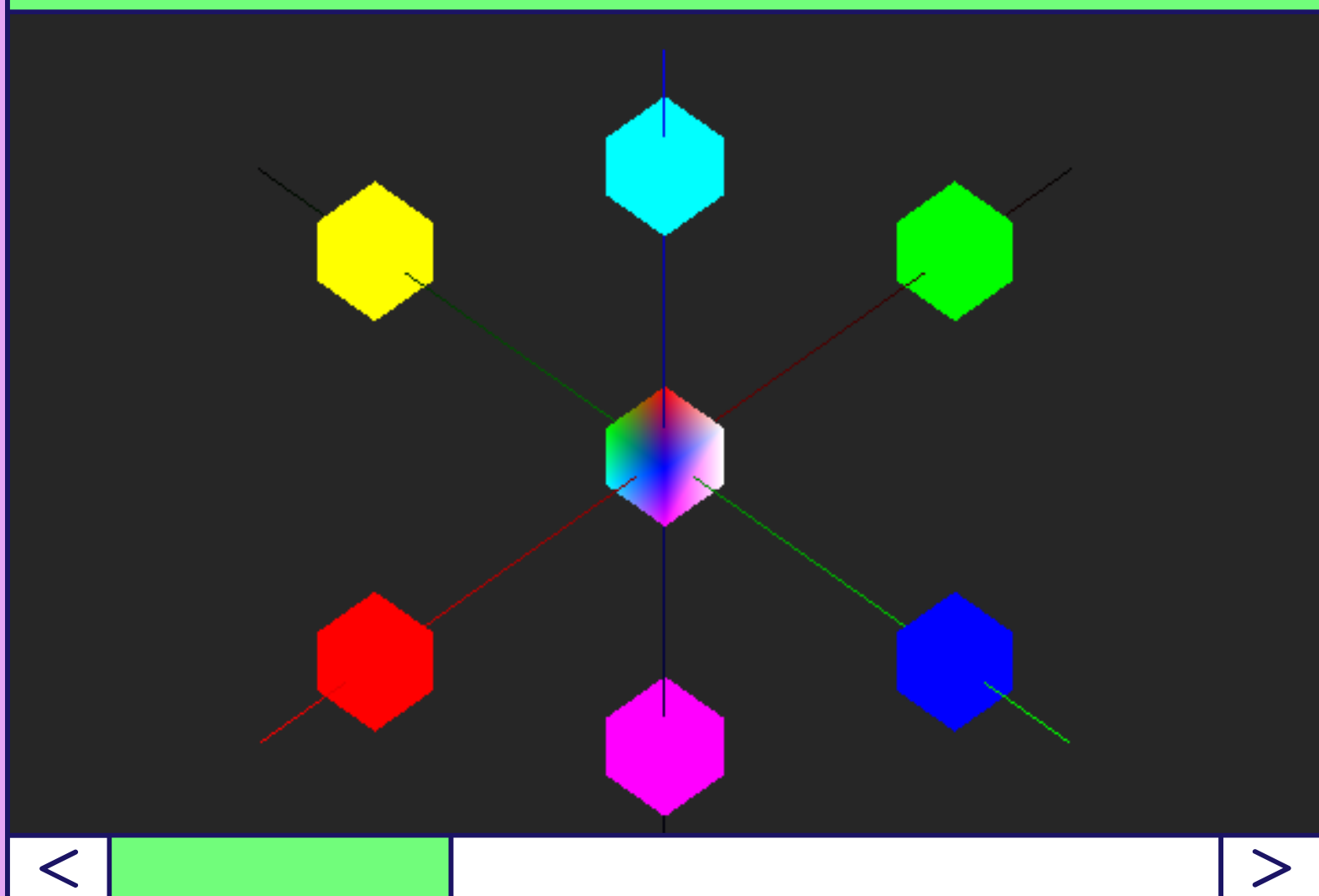
CÓMO ASIGNAR COORDENADAS DE TEXTURA

```
vertices = [  
#   positions      texCoords  
-0.5, -0.5, 0.0,  0, 0,  
 0.5, -0.5, 0.0,  1, 0,  
 0.5,  0.5, 0.0,  1, 1,  
-0.5,  0.5, 0.0,  0, 1]  
  
indices = [  
 0, 1, 2,  
 2, 3, 0]
```



EJEMPLOS EN GITHUE

✕ □ – examples/ex_projections.py



✕ □ – examples/ex_texture_boos.py

